UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2016/2017

SEPTEMBER EXAMINATION

UECS3253 WIRELESS APPLICATION DEVELOPMENT

SATURDAY, 10 SEPTEMBER 2016 TIME : 2.00 PM – 4.00 PM (2 HOURS)

BACHELOR OF SCIENCE (HONS) SOFTWARE ENGINEERING

Instruction to Candidates :

This question paper consists of THREE (3) questions in Section A and TWO (2) questions in Section B.

Section A: Answer ALL questions.

Section B: Answer ONE (1) question only.

Each question carries 25 marks.

Note: Should a candidate answer more than ONE (1) question in Section B, marks will only be awarded for the FIRST question in Section B in the order the candidate submits the answer.

Answer the questions in the answer booklet provided.

SECTION A

This section consists of **THREE (3)** questions. Answer ALL questions.

Q1. (a) Explain the following **FOUR (4)** features of the Android platform.

(i) Data storage. (3 marks)

(ii) Connectivity. (3 marks)

(iii) Messaging. (3 marks)

(iv) Tethering. (3 marks)

(b) Write the necessary XML code segments for the **AndroidManifest.xml** file of an Android app based for the specifications stated below. You may assume any appropriate resources id or name for the required attributes. The package name for this app is **com.chiakimhooi.duckie**.

(i) Define an activity named **DisplayDuckActivity**. This activity has **ListDucksActivity** as the hierarchical parent. (3 marks)

(ii) Define a service named **NotificationService**. (2 marks)

(c) Yoke Wai has been assigned the task to write code for implementing various features of a mobile app to communicate with external apps. One of the features he is implementing is to open a URL in a web browser.

(i) Write Java code segment to open an HTTP URL specified in a string named **url** in a web browser using an implicit intent. (5 marks)

(ii) Write XML code segment to specify the appropriate intent filter for the implicit intent in (i). (3 marks)

[Total : 25 marks]

Q2. Li Li and Pei Li are members of a mobile app development project to develop *Hotspots Everywhere*, an app which is available for both Android and iOS platforms that provides information about free wireless hotspots worldwide.

(a) Li Li is assigned the task to develop the functionality to edit and save user settings for this app. Table 2.1 shows the collection of key-value representing user settings that are required in this app.

Key	Example Value
showOpenHotspots	*false*
showWEPHotspots	*true*
colorScheme	OrangeDarken4

Table 2.1: Key-values

Write Java code segment to write the above key-values to a Shared Preference named **com.duck.hotspotseverywhere.SETTINGS**. Assume that the variables shown below have been declared and contain the appropriate values.

```
boolean showOpenHotspots;
boolean showWEPHotspots;
String colorScheme;
```

(7 marks)

(b) Pei Li is assigned the task to write Java code for implementing classes for data storage for the Android app.

Table 2.2 shows the structure of table named **hotspot** which is used to store data about wireless hotspots in a SQLite database named **Hotspot.db**. Figure 2.3 shows the class diagram for the class named **Hotspot** which will be used in the app where an instance of **Hotspot** represents a free wireless hotspot.

No.	Column Name	Data Type
1	id	Integer (Primary Key)
2	ssid	Text
3	networkKey	Text
4	encryptionType	Text

Table 2.2: Structure for *hotspot* table

Hotspot
-id : int -ssid : String -networkKey : String -encryptionType : String
+Hotspot() +getId() : int +getSsid() : String +getNetworkKey() : String +getEncryptionType() : String +setId(id : int) +setSsid(ssid : String) +setNetworkKey(networkKey : String) +setEncryptionType(encryptionType : String)

Figure 2.3: *Hotspot* class

Q2.(b) (Continued)

(i) Write Java code for the class named **HotspotDbOpenHelper**, which is an **SQLiteOpenHelper** class. Assume that the version of the database is **2**. The following methods must be implemented:

```
public HotspotDbOpenHelper(Context context);
public void onCreate(SQLiteDatabase db);
```

A **HotspotDBContract** class has also been declared as shown in Figure 2.4.

```
public final class HotspotDbContract {
    public HotspotDbContract() {}

    public static abstract class HotspotEntry
      implements BaseColumns {
        public static final String
          TABLE_NAME = "hotspot";
        public static final String
          COLUMN_NAME_SSID = "ssid";
        public static final String
          COLUMN_NAME_NETWORK_KEY = "networkKey";
        public static final String
          COLUMN_NAME_ENCRYPTION_TYPE = "encryptionType";
    }
}
```

Figure 2.4: *HotspotDBContract* class

(6 marks)

(ii) Write Java code segment to update a row in the **hotspot** table. Assume that the data to be updated is available in a **Hotspot** object referenced by a variable named **hotspot**. (6 marks)

(iii) Write Java code segment to read all rows in the **hotspot** table, sorted by **ssid** in ascending order and populate the values in an **ArrayList** of **Hotspot** objects. (6 marks)

[Total : 25 marks]

Q3. (a) Ai Li has taken up a project to develop an Android mobile app to provide information about koalas of a wildlife sanctuary in Queensland, Australia. This app is required to make a connection to an HTTP server over the Internet to download information of each individual koala before displaying it in an activity. Figure 3.1 shows a sample JSON data for one koala sent by the HTTP server to the mobile app.

```
{"id":28,"name":"Catty","gender":"Female","birthYear":2008}
```

Figure 3.1: Sample JSON object for a koala

(i) Assume that the app needs to make a HTTP GET request to the URL **http://koala.com.au/getkoalas** to obtain data for all koalas. Write Java code segment to make an **HttpUrlConnection** to the server to obtain the **InputStream** object for this connection. (7 marks)

(ii) Using the **InputStream** object in (i), write Java code segment to read the input stream and convert the data into a JSON array. (7 marks)

(iii) Using the JSON object in (ii), write Java code segment to process the JSON array and construct an **ArrayList** of **Koala** objects based on the data in the JSON array. The **Koala** class is as shown in Figure 3.2.

Koala
-id : int -name : String -gender : String -birthYear : int
+Koala() +getId() : int +getName() : String +getGender() : String +getBirthYear() : int +setId(id : int) +setName(name : String) +setGender(gender : String) +setBirthYear(birthYear : int)

Figure 3.2: *Koala* class

(4 marks)

(b) Xiao Hui has been asked to develop an Android app that involves getting the last known location of the user's device. Write Java code segment to obtain the last known location and open an appropriate map app using implicit intent. The location obtained should be specified in the data URI scheme in the implicit intent. (7 marks)

[Total : 25 marks]

SECTION B

This section consists of **TWO (2)** questions. Answer **ONE (1)** question only.

Q4. (a) Figure 4.1 shows the output of an activity of an Android app that displays a list of names using a **ListView** widget.

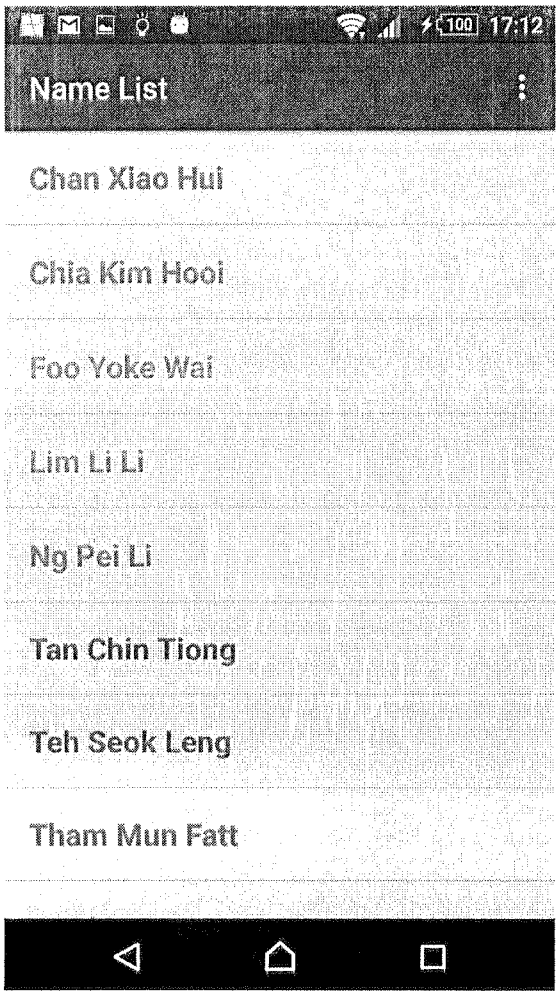

Figure 4.1: An activity with *ListView*

When the activity is created, the app loads the names from an **ArrayList** of strings to populate the **ListView**.

(i) Write XML code segment to define the **ListView** widget for the above activity. The resource **id** for the **ListView** should be assigned with the value **list_view**. (4 marks)

(ii) Write XML code to define the layout for a list item. The layout file is to be named **list_item.xml** and contains only a **TextView** widget. Set appropriate values for the **TextView** attributes to configure the alignment of the text as shown in Figure 4.1. The **TextView** should also have a height of **60dp** while the text size should be set to **20sp** and appear as **bold**. (5 marks)

Q4(a). (Continued)

(iii) Write Java code to implement the **OnItemLongClickListener** for the **ListView**. When an item is long-clicked, it opens an activity named **DisplayNameActivity**, passing the String value of the name in the clicked item as an extra using an **Intent** object. (9 marks)

(b) Figure 4.2 shows the output of an activity using a **LinearLayout** view group with a **ToggleButton** widget and an **ImageView** widget. Setting the **ToggleButton** to ***off*** will turn off the visibility of the **ImageView.** Otherwise, setting the **ToggleButton** to ***on*** will turn on the visibility of the **ImageView**.

Figure 4.2: An activity with *ToggleButton* and *ImageView*

Write Java code to implement the **OnCheckedChangeListener** for the **ToggleButton**. Assume that the resource **id** for the **ToggleButton** and **ImageView** are **toggle_picture** and **image** respectively. (7 marks)

[Total : 25 marks]

Q5. (a) Figure 5.1 shows a **DatePicker** widget that is shown as a dialog when a user taps on an **EditText** text field in an activity. Upon selecting a date from the **DatePicker** widget, the **EditText** text field will be filled with the selected date in the format ***yyyy-mm-dd***.

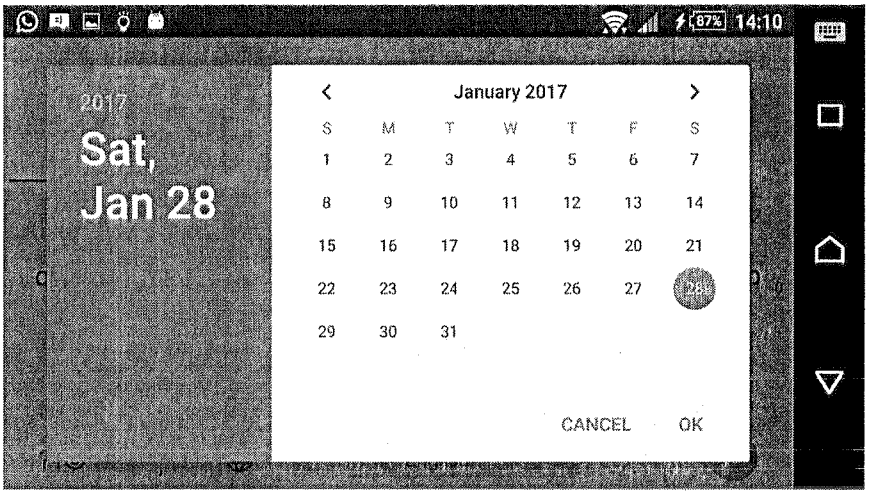

Figure 5.1: A *DatePicker* widget

In this scenario, a class named **DatePickerFragment** must be declared for displaying the **DatePicker** widget in a dialog fragment. The **DatePickerFragment** must have the following property and methods declared:

```
Calendar calendar;
public Dialog onCreateDialog(Bundle savedInstanceState);
public void onDateSet(DatePicker v, int y, int m, int d);
```

(i) Write Java code for the **DatePickerFragment** class. The property and methods shown above must be declared in the class definition.

The **onCreateDialog()** method is used show the **DatePicker** widget as a dialog. The initial date to be displayed on the **DatePicker** widget is provided by the date specified in the **calendar** property in the **DatePickerFragment** class.

The code for the **onDateSet()** method is used to fill up the **EditText** widget with the selected date when a date is selected. Assume that the **EditText** widget has the resource **id** set to **date**. (12 marks)

(ii) Write Java code segment to show how you would set the maximum allowed date for a **DatePicker** widget to *Friday, February 16, 2018*. (4 marks)

Q5. (Continued)

(b) Figure 5.2 shows the output of an activity with a **RelativeLayout** view group as its root.

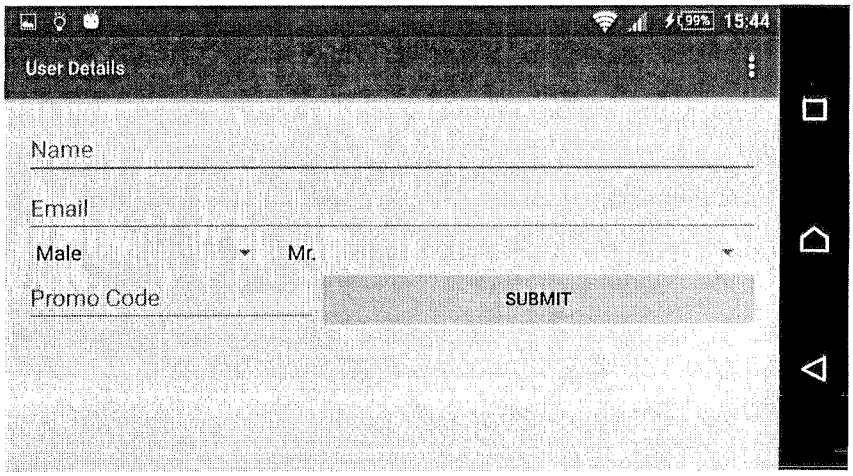

Figure 5.2: An activity using *RelativeLayout*

Write the XML code segment to add the view widgets as shown in Figure 5.2 to the **RelativeLayout** element for the above activity's layout. Assume that both the spinners in Figure 5.2 obtained their item values from the array resource **gender_array** and **salutation_array** respectively. Table 5.3 shows the values of the resource **id** that must be assigned to each view widget shown above.

View widget	Resource *id*
EditText (Name)	name
EditText (Email)	email
Spinner (Gender - Male, Female)	gender
Spinner (Salutation – Mr, Mrs, etc.)	salutation
EditText (Promo Code)	promo_code

Table 5.3: Resource *id* for view widgets

(9 marks)
[Total : 25 marks]